# Software Architectures Big Data

Exercise 05

*Submitted by Oluwatoose Oke*

## *Stage A — Today's system*

1. The detector keeps up, yet `missing` grows. Are the missing readings in the table? Why did the detector never see them? Read `PostgresSink` in `plant.py`, then think through this timeline: gateway 1 inserts rows that get the ids 100–114 and commits 100 ms later; gateway 2 inserts rows 115–129 and commits at once. A row gets its `id` when it is inserted, but others see it only when it is committed. Which rows does the detector see when it polls in between, and which `id` does it remember?

Answer:

From PostgressSink in plant.py, the readings do not appear to be incorrectly inserted in the table. The reason why the detector does not see them could have to do with transaction isolation.

When the detector polls, it runs SELECTS {...} FROM readings WHERE id > last_id. ORDER BY id.

A row gets its id when it is inserted, but others see it only when it is committed, so say gateway 1 inserts rows that get the ids 100–114 and commits 100 ms later while gateway 2 inserts rows 115–129 and commits at once. The detector would see the committed 129 and assign it as the last_id, but since rows from gateway 1 are not yet committed, the detector assumes them as missing. Even when the gateway inserts are finally committed,since the detector strictly queries WHERE id > 129, it skips rows 100 – 114 permanently, which is why we get missing readings in the model despite the rows existing in the database.

2. What happens during full plant scale-up:

```
(venv) toshay@toshay-VirtualBox:~/Documents/SBD3_Oke/Exercises/sbd-ais-ws26/Exc_5/guardian$ python3 baseline_
detector.py
[baseline] polling table readings for id > 18780 (Ctrl+C to stop)
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read        0  backlog       0 rows  delay   0.0 s  missing      0  anomalies 0
[baseline] read    1,830  backlog   2,655 rows  delay   4.5 s  missing      0  anomalies 0
[baseline] read    3,830  backlog   5,350 rows  delay   9.0 s  missing      0  anomalies 0
[baseline] read    5,830  backlog   8,180 rows  delay  13.7 s  missing      0  anomalies 0
[baseline] read    7,830  backlog  10,950 rows  delay  18.4 s  missing      0  anomalies 0
[baseline] read    9,830  backlog  13,660 rows  delay  22.9 s  missing      0  anomalies 0
[baseline] read   11,830  backlog  16,475 rows  delay  27.5 s  missing      0  anomalies 0
[baseline] read   13,830  backlog  19,215 rows  delay  32.1 s  missing      0  anomalies 0
[baseline] read   15,830  backlog  22,030 rows  delay  36.8 s  missing      0  anomalies 0
[baseline] read   17,830  backlog  24,755 rows  delay  41.3 s  missing      0  anomalies 0
[baseline] read   19,830  backlog  25,470 rows  delay  46.5 s  missing      0  anomalies 0
^C[baseline] stopped: read 19,830 readings, last id 38,802, 0 readings missing, 0 anomalies
(venv) toshay@toshay-VirtualBox:~/Documents/SBD3_Oke/Exercises/sbd-ais-ws26/Exc_5/guardian$
```

- Delay: Increases rapidly (~ +4.5s per poll cycle)
- Backlog: Grows significantly ( ~+2,700 rows per cycle)
- Missing: Remains at 0 during active running, but model latency becomes unusable

Reasoning: At 60 machines with 10 Hz (600 readings/sec) inserting into Postgres creates large write and lock contention. Scoring a reading takes ~3 ms per row, so a polling thread can process at most 1/0.003s ≈ 333 readings/s.

The plant sends 600 readings/s which exceeds the processing capacity, so the backlog and delay builds up infinitely.

# *Stage B - Comparing Optimization Options*

## Option 1: keep PostgreSQL, but with multiple polling

We split the input workload so multiple instances of baseline_detector can score concurrently instead of relying on a single poller process. Backlog on one group of machines does not prevent other workers from making progress on theirs.

### Requirement R1: Scale (100k readings/s) ⚠

> Each poller can process ≈ 333 readings/s. For a plant of 100K readings/s, you would require at least 301 detector instances to meet the scale requirement.
>
> Even with index support, so many polling workers executing frequent SELECT queries against a fast-growing table creates heavy database CPU and I/O lock contention.

### Requirement R2: Complete and ordered ⚠

> Because the lab gateways use `executemany()` in concurrent transactions, uncommitted rows with lower IDs can still be committed after a poller has already advanced its `last_id` past higher IDs. Workers can still skip readings unless extra tracking logic is introduced.

### Requirement R3: Fresh (<5s delay) ☒

> While polling can be sharded, all gateways perform INSERT operations into a single PostgresSQL table. At production scale (100,000 readings/s), database disk write I/O Write-Ahead-Logging can become heavy bottlenecks and cause unbounded backlogs.

### R4: Independent consumers ☒

> Adding additional services like a live dashboard and training data archives multiplies read queries against the database, increasing system load.

## Option 2: Streaming to Kafka (or another log-based platform)

Kafka satisfies each requirement through its distributed append-only log architecture, message partitioning, pull model, and consumer group offset tracking.

### Requirement R1: Scale (100k readings/s) ☑

Instead of a single database handling all writes and locks, Kafka producers (plant) append records directly to partition end-logs with minimal operational overhead. To scale throughput from 600 to 100,000 readings/sec, we can simply increase topic partitions and run additional detector instances in parallel within a consumer group.

### Requirement R2: Complete and ordered ☑

Kafka guarantees strict, immutable total ordering of records within a single partition. Setting the message key to `machine_id` during publish ensures that every reading for a particular machine is assigned to the exact same topic partition. Because a partition within a consumer group is consumed by exactly one worker instance at a time, every reading reaches the model in sequence.

### Requirement R3: Fresh (<5s delay ☑

Kafka uses a pull model where consumers send fetch requests directly to partition leaders and pull batches at their own maximum processing rate. Detector instances fetch data in real time as soon as it arrives at the broker log. Consumer lag remains near zero.

### R4: Independent consumers ☑

The detector service, live dashboard, and training data archive each belong to separate consumer groups (detector-group, dashboard-group, archive-group). Each service reads the stream independently at its own pace without locking partitions, altering data state, or generating query load on PostgreSQL.

## *Decision*

I select Option 2 (Apache Kafka) as it meets all requirements.

### Scalability Calculations:

- Model Inference Cost: 3 ms per reading.
- Single Instance Throughput: ≈ 333 readings/s.
- Lab Scale (600 readings/s):

$$Required\ Detector\ Instances = \frac{600}{333} = 2\ instances$$

Requires ≥ 2 partitions on the input Kafka topic

- Production Scale (100,000 readings/s):

$$Required\ Detector\ Instances = \frac{100{,}000}{333} = 301\ instances$$

Requires ≥ 301 partitions on the input Kafka topic

- The tipping point where a traditional database becomes inadequate is around 300 readings/second per processing thread, or whenever concurrent downstream requirements (training archives, live dashboards) begin competing for database resources.

# *Stage C - Challenge: Build the core*

## Command to Create Topic

```
docker exec -it kafka kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic readings --partitions 4 --replication-factor 1
```

## Implementation: plant.py

KafkaSink initialization settings:

*"acks: 1"*

Configures the broker to acknowledge a write as soon as the partition leader records it locally.

*"linger.ms: 5" (Micro-batching):*

Instructs the client thread to wait up to 5 milliseconds before emitting a TCP socket payload if the buffer isn't full, enabling batching.

*"queue.buffering.max.messages: 100000":*

Expands internal client RAM queues to absorb burst spikes, preventing plant.py main simulation threads from blocking or throwing memory buffer overflow errors

*Key Selection*

I used machine_id as the message key so that all readings of a specific machine is assigned to the exact same partition.

## Implementation: detector_service.py

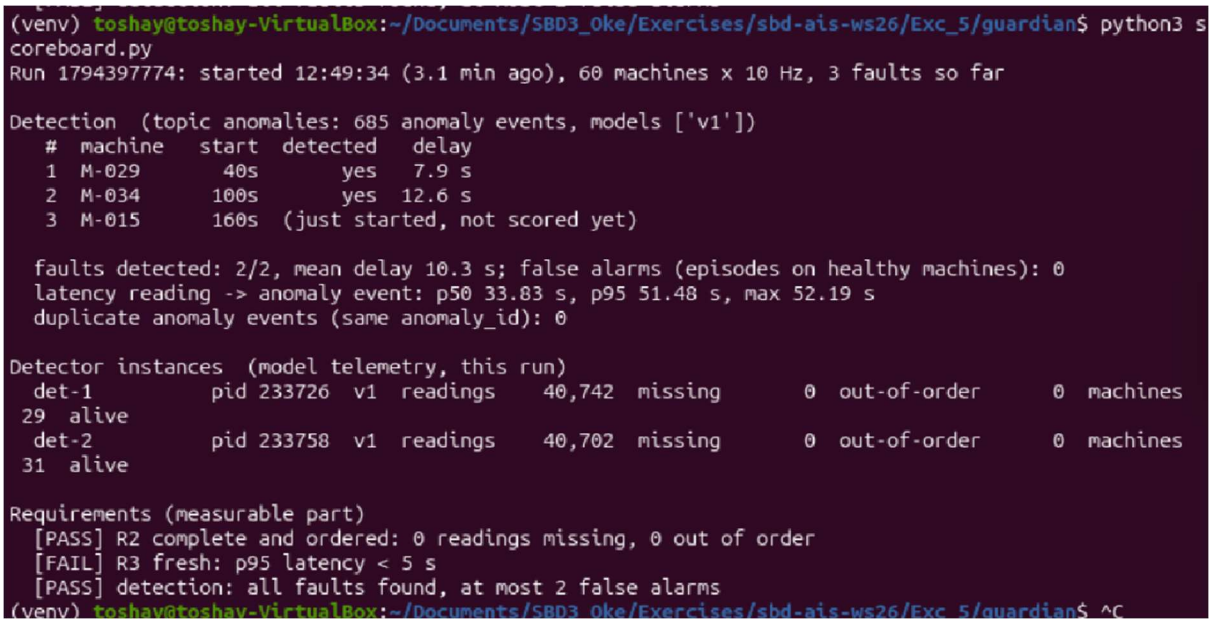
*auto.offset.reset = "latest":*

"latest" configures new or restarted consumer instances to consume strictly from the tail end of the log. For a real-time maintenance service, reading historical backlog from dead plant runs would inflate processing delays beyond the 5-second freshness target.

*Anomaly event keying (key=reading["machine_id"])*

This ensures that alerts for a specific machine is assigned to the exact same partition and in the right order of the anomalies topic.

## Scoreboard Output




Possible Explanation of R3 Failure:

The message `WARNING: plant X s behind schedule (sink too slow?)` appeared from time to time, starting at around 115s in one run and around 210s on another. A cause of this could be CPU core saturation. As I was running plant.py, PostgreSQL and 2 detector_service.py instances inside a single VM on my laptop, it is possible plant.py got starved of CPU resources.